Manual de Engenharia - AIDD Ouroboros

Manual de Engenharia • Visão Geral: Meta-Fluxo Ouroboros

> **Raio-X de Arquitetura do Ouroboros (Todas as Etapas)**
> Localização no pipeline: `[1. F1 ENTREVISTA] ➔ 2. SPEC ➔ 3. F2 ORQUESTRAÇÃO ➔ 4. HANDOFF ➔ 5. RUNNER`

📊 Grau Geral de Automação do Sistema

```
[████████████████████████████▒▒▒▒▒▒▒▒▒▒▒▒▒▒] 60% Determinístico | 40% Misto
```

- **60% Determinístico**: Validação de SPEC (JSON Schema), roteamento REA, scaffold de pipelines, instalação de deps, quality gates AST, assinatura HMAC-SHA256 e runner de DAG.
- **40% Misto**: Entrevista F1 (julgamento do agente + decisão humana), interpretação de NL, captura de rede na dissecação e scout GitHub (rede + ranking externo).

---

1. Índice das Etapas

#	Manual	Fase	Natureza dominante
01	Entrevista F1	F1	Humano + Agente
02	Semeadura da SPEC	F1	100% Determinístico
03	Validação da SPEC	F2	100% Determinístico
04	Roteamento REA	F2	100% Determinístico
05	Dissecação REA	F2	70% Det. / 30% Rede
06	Skill Creator	F2	85% Det. / 15% Rede
07	Pipeline Creator	F2	100% Determinístico
08	Dependency Resolver	F2	60% Det. / 40% Ambiente
09	Quality Gates	F2	100% Determinístico
10	Handoff Final	F2	100% Determinístico
11	Runner do Pipeline	Pós-F2	90% Det. / 10% Skills

---

2. Fluxo em Uma Linha

Pedido NL → Entrevista (aprovação humana) → **SPEC.json** → F2 autônoma: valida → REA (se houver alvo) → skills → scaffold → deps → audit → **handoff HMAC** → runner do DAG.

---

3. Convenções de Nomenclatura

Conceito	Significado
F1	Fase interativa (entrevista + SPEC)
F2	Fase autônoma (`--orchestrate`)
Exit 0/1/2/3	Sucesso / Erro técnico / Não aprovado / Dúvida (mini-entrevista)
Determinístico	Mesmo input → mesmo output, sem LLM
Probabilístico	Depende de LLM, julgamento ou rede externa

---

4. O que o Sistema Entrega, Para Quem e Como?

O que entrega	Para quem entrega	Como entrega (Formato / Meio)
SPEC aprovada	F2 (orquestrador)	Arquivo JSON: `SPEC_PIPELINE.json` com `approved: true`
Pipeline canônico	Usuário / Runner	Pasta: `OUT/aidd-pipelines/<nome>/` com DAG + runner
Handoffs assinados	Auditoria / próximo nó	JSON HMAC-SHA256: `OUT/handoffs/*.json`
Relatório de gates	CI / operador	Exit code 0 do `audit_runner` + log em stdout

Manual de Engenharia • Etapa 1: Entrevista F1

> **Raio-X de Arquitetura da Etapa 1 do Ouroboros**
> Localização no pipeline: `[1. ENTREVISTA] → 2. SPEC → 3. F2 ORQUESTRAÇÃO → 4. HANDOFF → 5. RUNNER`

📊 Grau Geral de Automação da Etapa

```
[████████████████▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒] 35% Determinístico | 65% Humano/Agente
```

- **35% Determinístico**: Semente da SPEC (`--from-nl`), schema JSON e limites de frontmatter.
- **65% Humano/Agente**: Interpretação do pedido em linguagem natural, uma pergunta por vez, palpite do agente e **aprovação explícita** (`approved: true`) que só o usuário pode dar.

---

1. Input da Etapa (O que entra e de onde vem)

Dado	Tipo	Exemplo	Origem
Pedido em linguagem natural	`string`	`"criar app de checklist simples"`	Usuário no chat ou `/ouroboros`
`--output-spec`	`Path`	`./SPEC_PIPELINE.json`	CLI (default: cwd)
`--goal`	`string`	`"provar fluxo de ponta a ponta"`	CLI / resposta da entrevista
Template da SPEC	`JSON`	`schemas/SPEC_PIPELINE.template.json`	Repositório

---

2. Processamento Passo a Passo (Microações e Natureza)

Ação 1.1: Reconhecimento do pedido

- **O que faz**: Identifica que o texto é um pedido de fabricação (pipeline/skill a partir de X) e ativa a trilha F1 — não pula para F2.
- **Natureza**: `40% Determinístico / 60% Probabilístico` (heurística simples + julgamento do agente).

Ação 1.2: Semeadura da SPEC

- **O que faz**: Executa `ouroboros_master.py --from-nl "<pedido>"`, gera `spec_id` (SHA-256) e grava template com `approved: false` e placeholders.
- **Natureza**: `100% Determinístico`.

Ação 1.3: Entrevista uma pergunta por vez (protocolo interview-me)

- **O que faz**: Pergunta, nesta ordem mínima: alvo (URL/APK/nenhum), sucesso observável, skills (`reuse` / `github-scout` / `create`), nome do pipeline, steps do DAG, instalação de deps. Cada pergunta leva um **palpite**.
- **Natureza**: `10% Determinístico / 90% Humano/Agente`.

Ação 1.4: Preenchimento e trava de aprovação

- **O que faz**: Escreve as respostas na SPEC; só após confirmação explícita grava `"approved": true`. Placeholders (`PREENCHER`, `placeholder-pipeline`) impedem a F2.
- **Natureza**: `50% Determinístico / 50% Humano` (a trava é sempre humana).

---

3. Scripts, Funções e Regras no Código

- **CLI de semente**: `ouroboros.py` • `--from-nl` / `--output-spec` / `--goal`.
- **Script mestre**: `aidd-skills/aidd-ouroboros-master/scripts/ouroboros_master.py` • função `cmd_init_spec`.
- **Template**: `schemas/SPEC_PIPELINE.template.json`.
- **Schema da SPEC**: `schemas/SPEC_PIPELINE.schema.json` (validado na Etapa 3).
- **Skill de conduta**: `aidd-ouroboros-master/SKILL.md` (F1) + skill de harness `interview-me`.
- **Regra de ouro**: F2 proibida sem `approved: true` e sem placeholders.

---

4. O que a Etapa Entrega, Para Quem e Como?

O que entrega	Para quem entrega	Como entrega (Formato / Meio)
SPEC template	Usuário / Agente	Arquivo JSON: `SPEC_PIPELINE.json` com `approved: false`
SPEC preenchida e aprovada	Etapa 3 (Validação F2)	Mesmo JSON com `approved: true` e `pipeline.name` real
Resumo da entrevista	Usuário (humano)	Mensagem no chat (linhas de confirmação)
Pedido original	Auditoria / F2	Campo `user_request` + `goal` dentro da SPEC

Manual de Engenharia • Etapa 2: Semeadura da SPEC

> **Raio-X de Arquitetura da Etapa 2 do Ouroboros**
> Localização no pipeline: `1. ENTREVISTA → [2. SEMEADURA DA SPEC] → 3. F2 ORQUESTRAÇÃO → 4. HANDOFF → 5. RUNNER`

📊 Grau Geral de Automação da Etapa

```
[████████████████████████████████████████████████] 100% Determinístico | 0% Probabilístico
```

- **100% Determinístico**: Hash do pedido, leitura de template e escrita de JSON — sem LLM, sem rede, sem aleatoriedade (apenas timestamp no `spec_id` / `created_at`).

---

1. Input da Etapa (O que entra e de onde vem)

Dado	Tipo	Exemplo	Origem
`--from-nl`	`string`	`"criar pipeline demo a partir de https://example.com"`	CLI do operador
`--output-spec`	`Path`	`./SPEC_PIPELINE.json`	CLI (default: `./SPEC_PIPELINE.json`)
`--goal`	`string`	`"provar fluxo"`	CLI (opcional)
Template	`JSON`	`schemas/SPEC_PIPELINE.template.json`	Repositório

---

2. Processamento Passo a Passo (Microações e Natureza)

Ação 2.1: Carregamento do template

- **O que faz**: Lê `SPEC_PIPELINE.template.json` (estrutura com placeholders e `approved: false`).
- **Natureza**: `100% Determinístico`.

Ação 2.2: Geração do identificador canônico

- **O que faz**: `spec_id = "spec-" + sha256(user_request + timestamp_utc)[:16]`.
- **Natureza**: `100% Determinístico`.

Ação 2.3: Preenchimento dos campos de cabeçalho

- **O que faz**: Grava `user_request`, `goal`, `created_at` (UTC) e força `approved: false`.
- **Natureza**: `100% Determinístico`.

Ação 2.4: Persistência e anúncio

- **O que faz**: Escreve JSON indentado no caminho de `--output-spec`, imprime a SPEC no stdout e a instrução da F1 no stderr.
- **Natureza**: `100% Determinístico`.

---

3. Scripts, Funções e Regras no Código

- **Função**: `ouroboros_master.py` • `cmd_init_spec(output, user_request, goal)`.
- **CLI espelho**: `ouroboros.py --from-nl` • despacha para o script da mestra.
- **Template**: `schemas/SPEC_PIPELINE.template.json`.
- **Regra**: nunca seta `approved: true`; nunca remove placeholders (papel exclusivo da Etapa 1).

---

4. O que a Etapa Entrega, Para Quem e Como?

O que entrega	Para quem entrega	Como entrega (Formato / Meio)
SPEC inicial	Usuário / agente (F1)	Arquivo JSON em `--output-spec` + cópia no stdout
Estado de bloqueio	Etapa 3 (validação F2)	`approved: false` (impede orquestração prematura)
Identificador estável	Audição / logs	Campo `spec_id`

Manual de Engenharia • Etapa 3: Validação da SPEC

> **Raio-X de Arquitetura da Etapa 3 do Ouroboros**
> Localização no pipeline: `1. ENTREVISTA → 2. SPEC → [3. VALIDAÇÃO DA SPEC] → 4. REA → 5. SKILLS → 6. PIPELINE → 7. DEPS → 8. GATES → 9. HANDOFF`

📊 Grau Geral de Automação da Etapa

```
[████████████████████████████████████████████████] 100% Determinístico | 0% Probabilístico
```

- **100% Determinístico**: JSON Schema (Draft7), checagem de `approved` e detecção de placeholders — fail-closed, sem LLM.

---

1. Input da Etapa (O que entra e de onde vem)

Dado	Tipo	Exemplo	Origem
`SPEC.json`	`Path` (CLI `--orchestrate`)	`./SPEC_PIPELINE.json`	Etapa 1 / 2
`--output-dir`	`Path`	`./ouroboros-out`	CLI (default: cwd)
`--resume`	`bool`	após exit 3	CLI
Chave HMAC	`env`	`OUROBOROS_CRYPTO_SECRET`	`.env` (para handoffs seguintes)

---

2. Processamento Passo a Passo (Microações e Natureza)

Ação 3.1: Existência e sintaxe JSON

- **O que faz**: Verifica arquivo no disco e `json.load`; erro → exit 1.
- **Natureza**: `100% Determinístico`.

Ação 3.2: Validação JsonSchema

- **O que faz**: `jsonschema.Draft7Validator` contra `schemas/SPEC_PIPELINE.schema.json`; cada violação formatada `caminho: mensagem` → exit 1.
- **Natureza**: `100% Determinístico`.

Ação 3.3: Travas de aprovação e placeholder

- **O que faz**: Se `approved != true` → exit 2. Se `user_request` contém `PREENCHER` ou `pipeline.name` for `placeholder-pipeline`/vazio → grava `pending_question.json` e exit 3.
- **Natureza**: `100% Determinístico`.

Ação 3.4: Inicialização do runtime de execução

- **O que faz**: Cria `output-dir/handoffs`, fixa raiz de estado (`ouroboros_master_state.json` + `pending_question.json` no out_dir), marca `status: RUNNING`.
- **Natureza**: `100% Determinístico`.

---

3. Scripts, Funções e Regras no Código

- **Função**: `ouroboros_master.py` • `orchestrate(spec_path, resume, out_dir)`.
- **Validadores**: `validate_spec` + `spec_is_placeholder`.
- **Schema**: `schemas/SPEC_PIPELINE.schema.json` (`additionalProperties: false`, `approved` boolean, `pipeline.steps` minItems 1).
- **Regra**: nenhuma etapa F2 roda se esta retornar exit 1, 2 ou 3.

---

4. O que a Etapa Entrega, Para Quem e Como?

O que entrega	Para quem entrega	Como entrega (Formato / Meio)
SPEC validada	Próxima etapa F2 (REA)	Sinalização por exit code 0 + estado em memória
State inicial	Operador / mestre	`OUT/ouroboros_master_state.json` (`status: RUNNING`)
Pergunta pendente (exit 3)	Usuário (mini-entrevista)	`OUT/pending_question.json`
Recusa de avanço (exit 2)	Operador	Mensagem em stderr + código 2

Manual de Engenharia • Etapa 4: Roteamento REA

> **Raio-X de Arquitetura da Etapa 4 do Ouroboros**
> Localização no pipeline: `3. VALIDAÇÃO SPEC → [4. ROTEAMENTO REA] → 5. DISSECAÇÃO → 6. SKILLS → 7. PIPELINE → 8. GATES`

📊 Grau Geral de Automação da Etapa

```
[████████████████████████████████████████████████] 100% Determinístico | 0% Probabilístico
```

- **100% Determinístico**: Classificação por extensão, prefixo de URL e keywords. Sem rede na classificação; o despacho apenas executa o especialista (que pode ter rede).

---

1. Input da Etapa (O que entra e de onde vem)

Dado	Tipo	Exemplo	Origem
`target.kind`	`enum`	`"web"` | `"apk"` | `"video"` | `"social"` | `"none"`	SPEC (Etapa 1)
`target.uri` / `path`	`string`	`https://example.com` ou `/app.apk`	SPEC
`--output-dir` (absoluto)	`Path`	`C:/.../OUT/rea`	Mestra F2
`--output-handoff` (absoluto)	`Path`	`C:/.../OUT/handoffs/01_rea_handoff.json`	Mestra F2

> Se `target.kind == "none"`, a etapa inteira é **SKIPPED** (projeto do zero).

---

2. Processamento Passo a Passo (Microações e Natureza)

Ação 4.1: Decisão de execução

- **O que faz**: Sem alvo → `SKIPPED`; com alvo → prossegue.
- **Natureza**: `100% Determinístico`.

Ação 4.2: Classificação da mídia

- **O que faz**: Mapeia alvo → dissecador: `.apk`/binário → `aidd-rea-app-android`; `.mp4`/YouTube → `aidd-rea-video-youtube`; `http(s)` → `aidd-rea-web-saas`; slides/pasta → `aidd-rea-social-carousel`.

- **Natureza**: `100% Determinístico`.

Ação 4.3: Despacho do especialista

- **O que faz**: `rea_router.py --target <uri> --output-dir <abs> --output-handoff <abs>` em subprocesso, cwd = raiz do repo.
- **Natureza**: `100% Determinístico` (orquestração); o trabalho pesado é da Etapa 5.

Ação 4.4: Pré-checagem de saída

- **O que faz**: Exige rc=0, existência do handoff e **verificação HMAC** do envelope da REA. Falha → `pending_question.json` + exit 3.
- **Natureza**: `100% Determinístico`.

---

3. Scripts, Funções e Regras no Código

- **Função da mestra**: `ouroboros_master.py` • `stage_rea`.
- **Script**: `aidd-skills/aidd-rea-router/scripts/rea_router.py` • `classify_target`, `route_and_execute`.
- **Regra de caminho**: paths de saída **absolutos** (path relativo vaza para `aidd-skills/<skill>/...`).
- **Gates**: handoff deve verificar `verify_handoff_envelope == True`.

---

4. O que a Etapa Entrega, Para Quem e Como?

O que entrega	Para quem entrega	Como entrega (Formato / Meio)
Despacho	Dissecador especialista (Etapa 5)	Subprocesso CLI com alvo + paths absolutos
Estado da etapa	Mestra / operador	`stages.rea = {status, handoff, hmac}` no state
Handoff da REA	Mestra (verificação) e downstream	`OUT/handoffs/01_rea_handoff.json` (HMAC)
SKIP legítimo	Mestra	`stages.rea = {status: SKIPPED}` se `kind=none`

Manual de Engenharia • Etapa 5: Dissecação REA

> **Raio-X de Arquitetura da Etapa 5 do Ouroboros**
> Localização no pipeline: `3. VALIDAÇÃO SPEC → 4. ROTEAMENTO → [5. DISSECAÇÃO REA] → 6. SKILLS → 7. PIPELINE → 8. GATES`

📊 Grau Geral de Automação da Etapa

```
[████████████████████████████████████▒▒▒▒▒▒▒▒▒▒▒▒] 70% Determinístico | 30% Rede/Ambiente
```

- **70% Determinístico**: Extração de tokens, manifests, hashing SHA-256, escrita de relatórios e selagem HMAC.
- **30% Rede/Ambiente**: Fetch de URL, download de vídeo, OCR local (tesseract) e fallbacks quando binários/rede falham.

---

1. Input da Etapa (O que entra e de onde vem)

Dado	Tipo	Exemplo	Origem
Alvo de engenharia reversa	`string`	`https://example.com`	Etapa 4 (router)
`--output-dir`	`Path` abs.	`OUT/rea`	Mestra
`--output-handoff`	`Path` abs.	`OUT/handoffs/01_rea_handoff.json`	Mestra
Binários opcionais	`system`	`ffmpeg`, `yt-dlp`, `tesseract`	Ambiente
Chave HMAC	`env`	`OUROBOROS_CRYPTO_SECRET`	`.env`

---

2. Processamento Passo a Passo (Microações e Natureza)

Ação 5.1: Abertura e normalização do alvo

- **O que faz**: Conforme tipo: HTTP fetch, abertura de APK/zip, leitura de imagens ou aquisição de vídeo/metadata.
- **Natureza**: `50% Determinístico / 50% Rede` (web/vídeo) ou `100% Determinístico` (APK local).

Ação 5.2: Extração de evidências (microação por família)

- **O que faz**:
  - **Web**: tokens de design, `conteudo_tecnico.md`, `estrutura_ux.md`, `MANIFEST_REA.json`.

    - **APK**: package, permissões, activities, endpoints + manifesto.
    - **Vídeo**: metadata, metodologia, evidências visuais.
    - **Social**: OCR slide a slide, paleta, hook/CTA, manifesto.
- **Natureza**: `90% Determinístico / 10% Fallback de rede ou OCR`.

Ação 5.3: Selo do envelope REA

- **O que faz**: `emit_handoff_hmac` → `criar_envelope_handoff` com ponteiros `{path, sha256, tamanho_bytes}` e `domain_metadata` do alvo.
- **Natureza**: `100% Determinístico` (HMAC-SHA256 + SHA-256 de arquivos).

Ação 5.4: Publicação no workspace da F2

- **O que faz**: Escreve artefatos sob `OUT/rea/**` e handoff em `OUT/handoffs/01_rea_handoff.json` (caminhos absolutos).
- **Natureza**: `100% Determinístico`.

---

3. Scripts, Funções e Regras no Código

- **Web**: `aidd-rea-web-saas/scripts/web_reengineer.py` • `dissect_web_application`, `emit_handoff_hmac`.
- **APK**: `aidd-rea-app-android/scripts/app_reengineer.py` • `dissect_mobile_or_binary_app`.
- **Vídeo**: `aidd-rea-video-youtube/scripts/video_reengineer.py` • `generate_evidence_manifest`.
- **Social**: `aidd-rea-social-carousel/scripts/social_reengineer.py` • `dissect_social_carousel`, `ocr_image_file`.
- **Motores**: `handoff_crypto.criar_envelope_handoff`, `credentials.carregar_env_local`.
- **Regra**: envelope canônico `{body, seal}` — proibido legado flat `handoff-v1` sem assinatura.

---

4. O que a Etapa Entrega, Para Quem e Como?

O que entrega	Para quem entrega	Como entrega (Formato / Meio)
Tokens / relatórios / manifesto	Usuário e etapas seguintes	Arquivos em `OUT/rea/**` (JSON + Markdown)
Handoff assinado da REA	Mestra (Etapa 4/6) e pipelines	`OUT/handoffs/01_rea_handoff.json` (HMAC verificável)
Ponteiros com hash	Auditoria forense	`body.artifacts_pointers[*].sha256`
Dados do domínio	Downstream / humano	`body.domain_metadata` (ex.: permissões do APK)

Manual de Engenharia • Etapa 6: Skill Creator

> **Raio-X de Arquitetura da Etapa 6 do Ouroboros**
> Localização no pipeline: `5. DISSECAÇÃO → [6. SKILL CREATOR] → 7. PIPELINE CREATOR → 8. DEPS → 9. GATES`

📊 Grau Geral de Automação da Etapa

```
[████████████████████████████████████▒▒▒▒▒▒▒▒▒▒▒▒] 85% Determinístico | 15% Rede (GitHub)
```

- **85% Determinístico**: Checagem de existência local, scaffold enterprise a partir de templates, auditoria de gates da skill.
- **15% Rede**: GitHub Scout (`gh` /API) com cache e ranking externo — só quando `mode = github-scout` e a skill não existe localmente.

---

1. Input da Etapa (O que entra e de onde vem)

Dado	Tipo	Exemplo	Origem
`skills_required[]`	`array`	`[{name, role, mode}]`	SPEC (Etapa 1)
`mode`	`enum`	`reuse` | `github-scout` | `create`	SPEC
`github_query`	`string`	`"checklist tdd"`	SPEC (se scout)
`--output-dir`	`Path`	`aidd-skills/` do repo	Mestra
Cache GitHub	`JSON`	`.cache/github_scout_cache.json`	Disco (TTL 7 dias)

---

2. Processamento Passo a Passo (Microações e Natureza)

Ação 6.1: Resolução por reuso

- **O que faz**: Se `aidd-skills/<name>/SKILL.md` existe → `action: reuse`, segue sem criar nada.
- **Natureza**: `100% Determinístico`.

Ação 6.2: Criação proprietária (mode create)

- **O que faz**: `skill_creator.py --nome <name> --output-dir` gera anatomia enterprise (scripts, references, templates, AGENTS, RULES, sync, dependencies, comandos).
- **Natureza**: `100% Determinístico` (templates fixos).

Ação 6.3: GitHub Scout (mode github-scout)

- **O que faz**: Busca repositórios por estrelas/licença via `gh` ou REST, usa cache local, refatora no prefixo canônico `aidd-<slug>`.
- **Natureza**: `40% Determinístico / 60% Rede` (resultado depende do GitHub no momento).

Ação 6.4: Auditoria da skill e registro de estado

- **O que faz**: Roda `--audit` da skill gerada; grava `state.skills[name] = {action, status}`. Falha → exit 3 + `pending_question.json`.
- **Natureza**: `100% Determinístico`.

---

3. Scripts, Funções e Regras no Código

- **Função da mestra**: `ouroboros_master.py` • `stage_skills`, `resolve_skill`.
- **Factory**: `aidd-orchestrate-skill-creator/scripts/skill_creator.py` • `create_canonical_skill`, `scout_and_canonize_skill`, `emit_handoff_envelope`.
- **Leis**: micro-frontmatter ≤ 25 palavras; dual-spec SKILL+AGENTS; autocontenção; **prioridade local → GitHub → create**; proibição de criação ad-hoc fora da factory.

---

4. O que a Etapa Entrega, Para Quem e Como?

O que entrega	Para quem entrega	Como entrega (Formato / Meio)
Skills prontas (reuso/novas)	Etapa 7 (pipeline) e harness (após sync)	Diretórios `aidd-skills/<nome>/`
Estado por skill	Operador / mestre	`stages.skills` + `state.skills` no state
Handoff da factory (quando cria)	Downstream	`HANDOFF.json` canônico HMAC na pasta da skill
Dúvida de criação (exit 3)	Usuário	`pending_question.json`

Manual de Engenharia • Etapa 7: Pipeline Creator (Scaffold)

> **Raio-X de Arquitetura da Etapa 7 do Ouroboros**
> Localização no pipeline: `6. SKILLS → [7. PIPELINE CREATOR] → 8. DEPS → 9. GATES → 10. HANDOFF`

📊 Grau Geral de Automação da Etapa

```
[████████████████████████████████████████████████] 100% Determinístico | 0% Probabilístico
```

- **100% Determinístico**: Scaffold a partir de templates + reescrita do `PIPELINE.json` com o DAG exato da SPEC. Sem rede e sem LLM nesta etapa.

---

1. Input da Etapa (O que entra e de onde vem)

Dado	Tipo	Exemplo	Origem
`pipeline.name`	`string`	`"e2e-real"`	SPEC (Etapa 1)
`pipeline.steps[]`	`array`	`[{id, skill, depends_on}]`	SPEC
`--output-dir`	`Path`	`OUT/` (cwd do scaffold)	Mestra
Skills referenciadas	`nomes`	`aidd-rea-web-saas`	Etapa 6 / Almoxarifado

---

2. Processamento Passo a Passo (Microações e Natureza)

Ação 7.1: Validação do DAG mínimo

- **O que faz**: Exige `name` e `steps` com pelo menos um `skill` válido. Sem isso → exit 3.
- **Natureza**: `100% Determinístico`.

Ação 7.2: Scaffold canônico

- **O que faz**: `pipeline_creator.py --scaffold-pipeline <nome> --steps <csv-de-skills>` cria `OUT/aidd-pipelines/<slug>/` com README, AGENTS, RULES, LICENSE, PIPELINE.json, state.json, run_pipeline.py, context/, templates/, references/.
- **Natureza**: `100% Determinístico`.

Ação 7.3: Reescrita rica do DAG

- **O que faz**: Sobrescreve `PIPELINE.json` com IDs, descrições e `depends_on` originais da SPEC (o CSV do CLI só aciona o scaffold).
- **Natureza**: `100% Determinístico`.

Ação 7.4: Publicação no state da F2

- **O que faz**: `stages.pipeline = {status: OK, path: <dir>}`. Path ausente pós-scaffold → exit 3.
- **Natureza**: `100% Determinístico`.

---

3. Scripts, Funções e Regras no Código

- **Função da mestra**: `ouroboros_master.py` • `stage_pipeline`.
- **Factory**: `pipeline_creator.py` • `scaffold_autonomous_pipeline`, CLI `--scaffold-pipeline`.
- **Regras**: anatomia enterprise completa (circuit breaker); encapsulamento em pasta própria; nunca skills soltas na raiz; runner padrão de `templates/RUN_PIPELINE_TEMPLATE.py`.

---

4. O que a Etapa Entrega, Para Quem e Como?

O que entrega	Para quem entrega	Como entrega (Formato / Meio)
Pipeline encapsulado	Operador + Etapa 8 (deps) + Etapa 11 (runner)	Pasta `OUT/aidd-pipelines/<slug>/`
DAG canônico	Runner / humanos	`PIPELINE.json` com nós e `depends_on`
Runner	Execução pós-F2	`run_pipeline.py` (+ `state.json` INITIALIZED)
Estado F2	Mestra	`stages.pipeline.status = "OK"`

Manual de Engenharia • Etapa 8: Dependency Resolver

> **Raio-X de Arquitetura da Etapa 8 do Ouroboros**
> Localização no pipeline: `7. PIPELINE CREATOR → [8. DEPENDENCY RESOLVER] → 9. GATES → 10. HANDOFF`

📊 Grau Geral de Automação da Etapa

```
[████████████████████████▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒] 60% Determinístico | 40% Ambiente/Rede
```

- **60% Determinístico**: Leitura e validação estrita dos manifests `dependencies.json`, checagem de pacotes/binários locais.
- **40% Ambiente/Rede**: Instalação real via pip/uv (estado da máquina e download de pacotes).

---

1. Input da Etapa (O que entra e de onde vem)

Dado	Tipo	Exemplo	Origem
`install_dependencies`	`bool`	`true` / `false`	SPEC (Etapa 1)
`dependencies.json` do pipeline	`JSON`	`OUT/aidd-pipelines/<slug>/dependencies.json`	Etapa 7
`dependencies.json` das skills	`JSON`	`aidd-skills/<name>/dependencies.json`	Repo / Etapa 6
Flags	CLI	`--manifest <p> --install --yes`	Mestra (sempre `--yes`)

> Se `install_dependencies == false` → etapa **SKIPPED**.

---

2. Processamento Passo a Passo (Microações e Natureza)

Ação 8.1: Decisão de execução

- **O que faz**: SPEC pediu skip → `SKIPPED`; senão coleta manifests (pipeline + skills da SPEC).
- **Natureza**: `100% Determinístico`.

Ação 8.2: Resolução estática do manifesto

- **O que faz**: `DependencyResolver(manifest)` valida estrutura, versões pinadas, pacotes Python e binários de sistema.

- **Natureza**: `100% Determinético`.

Ação 8.3: Instalação não-interativa

- **O que faz**: `install(non_interactive=True)` executa o gerenciador de pacotes. rc≠0 → estado FAILED + exit 3 (pergunta: continuar sem instalar? ajustar versão?).
- **Natureza**: `30% Determinístico / 70% Ambiente+Rede`.

Ação 8.4: Registro do resultado

- **O que faz**: `stages.dependencies = {status, count|manifest, rc}` no state da F2.
- **Natureza**: `100% Determinístico`.

---

3. Scripts, Funções e Regras no Código

- **Função da mestra**: `ouroboros_master.py` • `stage_dependencies`.
- **Motor**: `motores/governanca/dependency_resolver.py` • `DependencyResolver.check` / `.install`.
- **CLI**: `python motores/governanca/dependency_resolver.py --manifest <path> --install --yes`.
- **Regras**: fail-closed em manifesto inválido; instalação exige confirmação não-interativa (`--yes`); zero stubs.

---

4. O que a Etapa Entrega, Para Quem e Como?

O que entrega	Para quem entrega	Como entrega (Formato / Meio)
Pacotes instalados	Execução futura dos scripts (etapas/runner)	Ambiente Python (venv/site-packages)
Estado da etapa	Mestra / operador	`stages.dependencies` no `state.json` da F2
Dúvida de ambiente (exit 3)	Usuário	`pending_question.json`
Skip legítimo	Mestra	`stages.dependencies = {status: SKIPPED}`

Manual de Engenharia • Etapa 9: Quality Gates (Audit Runner)

> **Raio-X de Arquitetura da Etapa 9 do Ouroboros**
> Localização no pipeline: `7. PIPELINE → 8. DEPS → [9. QUALITY GATES] → 10. HANDOFF FINAL`

📊 Grau Geral de Automação da Etapa

```
[████████████████████████████████████████████████] 100% Determinístico | 0% Probabilístico
```

- **100% Determinístico**: AST, contagem de palavras, checagens de filesystem e baseline — mesma árvore de código ⇒ mesmo resultado.

---

1. Input da Etapa (O que entra e de onde vem)

Dado	Tipo	Exemplo	Origem
Raiz do repositório	`Path`	cwd ou `AIDD_OUROBOROS_ROOT`	Ambiente
`--skip-pytest`	`bool`	presente na F2	Mestra / CLI
Baseline de violações	`JSON`	`motores/governanca/gate_baseline.json`	Repo
Código, skills, pipelines	filesystem	`aidd-skills/**`, `aidd-pipelines/**`	Repo

---

2. Processamento Passo a Passo (Microações e Natureza)

Ação 9.1: Gate AST / Zero-stubs

- **O que faz**: Varre `.py` com `gate_ast.py`; reprova sintaxe inválida e `except: pass` / stubs, respeitando `gate_baseline.json`.
- **Natureza**: `100% Determinístico`.

Ação 9.2: Gate Micro-frontmatter

- **O que faz**: `description` de cada `SKILL.md` ≤ 25 palavras.
- **Natureza**: `100% Determinístico`.

Ação 9.3: Gate AGENTS.md fractal

- **O que faz**: Existe e respeita limite de palavras do projeto.

- **Natureza**: `100% Determinístico`.

Ação 9.4: Gate anatômico (Circuit Breaker)

- **O que faz**: Reprova skills/pipelines anêmicos (ex.: `templates/` vazio = criação ad-hoc).
- **Natureza**: `100% Determinístico`.

Ação 9.5: Pytest (opcional)

- **O que faz**: Em F2 costuma vir `--skip-pytest`; no CI/`ouroboros --audit` completo roda a suíte.
- **Natureza**: `100% Determinístico` (execução local).

---

3. Scripts, Funções e Regras no Código

- **Função da mestra**: `ouroboros_master.py` • `stage_audit`.
- **Orquestrador**: `motores/governanca/audit_runner.py`.
- **AST**: `motores/governanca/gate_ast.py` • `scan_path`, baseline.
- **Regras do projeto**: zero stubs; micro-frontmatter; governança fractal; anatomia enterprise obrigatória.
- **Estado F2**: rc=0 → `stages.audit=OK` e F2 pode fechar SUCCESS; rc≠0 → F2 PARTIAL / exit 1.

---

4. O que a Etapa Entrega, Para Quem e Como?

O que entrega	Para quem entrega	Como entrega (Formato / Meio)
Veredito dos gates	Mestra / CI / operador	Exit code 0 ou 1 + log `[OK]` / `[FALHA]` em stdout
Estado do gate	Handoff final (Etapa 10)	`stages.audit = {status, rc}` → define SUCCESS vs PARTIAL
Evidência de conformidade	Revisor humano / auditoria	Relatório textual da execução do `audit_runner`

Manual de Engenharia • Etapa 10: Handoff Final (Envelope HMAC)

> **Raio-X de Arquitetura da Etapa 10 do Ouroboros**
> Localização no pipeline: `8. DEPS → 9. GATES → [10. HANDOFF FINAL] → 11. RUNNER`

📊 Grau Geral de Automação da Etapa

```
[████████████████████████████████████████████████] 100% Determinístico | 0% Probabilístico
```

- **100% Determinístico**: SHA-256 dos artefatos apontados + HMAC-SHA256 canônico (JSON ordenado). Sem chave → não emite (fail-closed).

---

1. Input da Etapa (O que entra e de onde vem)

Dado	Tipo	Exemplo	Origem
SPEC em memória	`dict`	campos `spec_id`, `user_request`, `pipeline`	Etapa 3
State da F2	`JSON`	`OUT/ouroboros_master_state.json`	Etapas 3–9
`output-dir`	`Path`	`OUT/handoffs/`	CLI
Chave HMAC	`env`	`AIDD_HANDOFF_KEY` | `OUROBOROS_HMAC_SECRET` | `OUROBOROS_CRYPTO_SECRET`	`.env` / ambiente
Ponteiros de artefato	`paths`	`state_path`, `pipeline_dir`	State (se existirem em disco)

---

2. Processamento Passo a Passo (Microações e Natureza)

Ação 10.1: Resolução da chave

- **O que faz**: `ensure_hmac_env()` carrega `.env` e lê a variável de segredo. Ausente → `RuntimeError` (não assina envelope falso).
- **Natureza**: `100% Determinístico`.

Ação 10.2: Montagem do payload

- **O que faz**: Define `status` (`SUCCESS` se audit OK, senão `PARTIAL`), hash SHA-256 dos artefatos existentes e metadados da SPEC/estágios.

- **Natureza**: `100% Determinístico`.

Ação 10.3: Selo do envelope

- **O que faz**: `criar_envelope_handoff` → `selar(body)` com HMAC-SHA256; espelha `payload` + `hmac_sha256` no topo (retrocompat).
- **Natureza**: `100% Determinístico`.

Ação 10.4: Persistência e limpeza

- **O que faz**: Grava `OUT/handoffs/master_final_handoff.json`; se audit OK, remove `pending_question.json` residual.
- **Natureza**: `100% Determinético`.

> **Paralelo da Etapa 4/5:** o handoff da REA (`01_rea_handoff.json`) usa o mesmo motor e é **verificado** antes desta etapa.

---

3. Scripts, Funções e Regras no Código

- **Função da mestra**: `ouroboros_master.py` • `emit_final_handoff`, `ensure_hmac_env`.
- **Motor cripto**: `motores/governanca/handoff_crypto.py` • `criar_envelope_handoff`, `selar`, `abrir`, `verify_handoff_envelope`.
- **Estrutura**: `envelope_format`, `body`, `seal`, espelho `payload` / `hmac_sha256`.
- **Regra**: pointer architecture — entrega ponteiros + hash, não despeja binários no chat.

---

4. O que a Etapa Entrega, Para Quem e Como?

O que entrega	Para quem entrega	Como entrega (Formato / Meio)
Envelope final da fabricação	Usuário / auditoria / próximo pipeline (Lego)	`OUT/handoffs/master_final_handoff.json` (JSON HMAC)
Status da corrida	Operador	Campo `status` (SUCCESS/PARTIAL) + `spec_id` no stdout
Verificabilidade	Qualquer consumidor	`verify_handoff_envelope(path) == True`
State fechado	Operador / resume	`ouroboros_master_state.json` com `finished_at`

Manual de Engenharia • Etapa 11: Runner do Pipeline Gerado

> **Raio-X de Arquitetura da Etapa 11 do Ouroboros**
> Localização no pipeline: `1..10 F2 (FABRICAÇÃO) ➔ [11. RUNNER] ➔ entrega em execução`

📊 Grau Geral de Automação da Etapa

```
[▓▓▓
```

Ação 11.2: Ordenação topológica do DAG

- **O que faz**: Ordena por `depends_on`; detecta ciclo → erro.
- **Natureza**: `100% Determinístico`.

Ação 11.3: Controle de estado e cache de inputs

- **O que faz**: `state.json` por `run_id`; `inputs_hash` (SHA-256 do input + estrutura do nó); nó COMPLETED com hash igual e sem `--force` → skip.
- **Natureza**: `100% Determinístico`.

Ação 11.4: Resolução e execução do nó

- **O que faz**: Lê `entrypoint` e `cli_contract.input_flag/output_flag`; monta comando; `subprocess` com cwd/workspace; rc≠0 → FAILED e aborta.
- **Natureza**: `95% Determinístico / 5% Mistura` (depende da skill invocada).

Ação 11.5: Fecho do workspace

- **O que faz**: Todos OK → `state.status = COMPLETED`, exit 0.
- **Natureza**: `100% Determinístico`.

---

3. Scripts, Funções e Regras no Código

- **Template**: `aidd-orchestrate-pipeline-creator/templates/RUN_PIPELINE_TEMPLATE.py`.
- **Funções**: `resolve_repo_root`, `resolve_skill_execution`, `topological_sort`, `execute_pipeline`.
- **Regras**: contrato `dependencies.json` + `cli_contract`; sem globbing de `*.py`; zero stubs; handoffs de nó quando a skill emite.
- **Contrato de testes**: `tests/test_entrypoints.py` garante entrypoint e flags existentes em todas as skills canônicas.

---

4. O que a Etapa Entrega, Para Quem e Como?

O que entrega	Para quem entrega	Como entrega (Formato / Meio)
Estado da execução	Operador / CI	`WORKSPACE/state.json` (`COMPLETED` / `FAILED`)
Saídas por nó	Downstream / humano	`WORKSPACE/<node_id>/`
Código de saída	Shell / pipeline de deploy	Exit 0 = todos os nós OK
Evidência de DAG real	Teste E2E	Ex.: `aidd-pipelines/_e2e-smoke` com HMAC por nó

> **Nota de operação:** a fabricação (F2) e a execução (runner) são separadas. O trabalho pesado de ponta a ponta comprovado hoje: **F2 da mestra** + runners com inputs reais (`_e2e-smoke`).